

Algorithmes gloutons II

Gestion événementielle

1. Préambule



1.1 Rappels sur les tris

Pour trier une liste A en Python, on procède de la sorte :

```
A = [1, 10, 4, 11, 13, 8, 6]
A.sort() # Trie la liste A, elle ne renvoie rien !
B = sorted(A) # Renvoie une copie de la liste A triée, A n'est pas modifiée !
```

En Python, les éléments sont par défaut triés dans l'ordre croissant. On peut changer cet ordre, notamment pour des objets plus complexes, grâce à l'attribut `key` et `reverse`:

```
A = [1, 10, 4, 11, 13, 8, 6]
A.sort() # Trié dans l'ordre croissant
A.sort(reverse=True) # Trié dans l'ordre décroissant
A.sort(key=lambda u : -u) # Aussi trié dans l'ordre décroissant
```

Pour visualiser l'ordre obtenu avec une clef de tri, il faut l'appliquer sur chaque élément, trier la liste, et revenir aux valeurs précédentes.

1.2 Remarques et astuces sur les tris

Note

En Python, les chaînes de caractères, n-uplets et les listes, sont triées dans l'ordre lexicographique.

Dans ce dernier, il est en général assez lent de trier des n-uplets selon des composantes précises. Pour y remédier, une astuce c'est de construire une liste d'indices, et de trier cette dernière :

```
A = [(1, 8, -5), (11, 19, 14), (10, 10, 10), (9, 1, 4)]  
indices = list(range(len(A)))  
indices.sort(key=lambda j : (A[j][1], A[j][2]))
```

2. Gestion événementielle

2.1 Rappel

On rappelle qu'un **algorithme glouton** est un algorithme qui fait le choix le plus optimal à chaque étape.

Par la suite, nous allons voir différents problèmes où nous devons calculer une valeur sur la base des événements qui se produisent.

2.2 Problème A

On s'intéresse au problème suivant :

Problème A

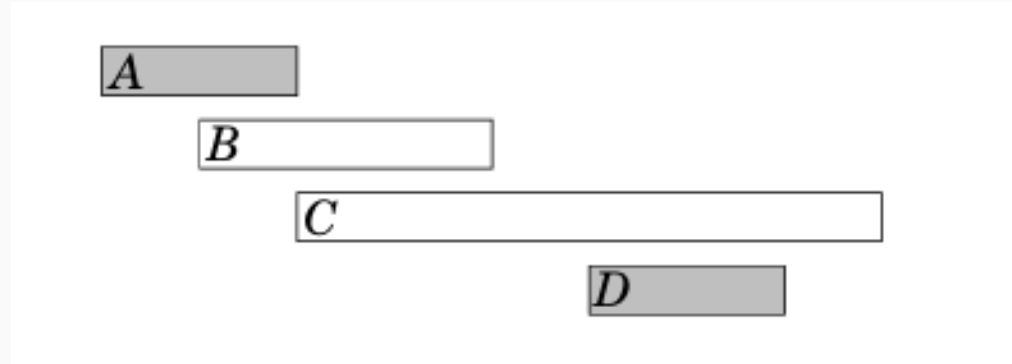
Si on a n événements avec leurs début et fin, trouver un emploi du temps qui inclut le plus d'événements possibles.

Nous allons prendre un exemple, avec la liste d'événements suivante:

Événement	Début	Fin
A	1	3
B	2	5
C	3	9
D	6	8

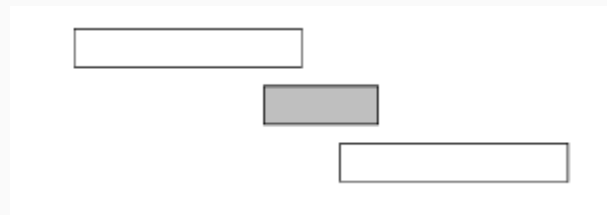
Les durées sont comptées par unité de temps ...

2.3 Problème A - Solution 1



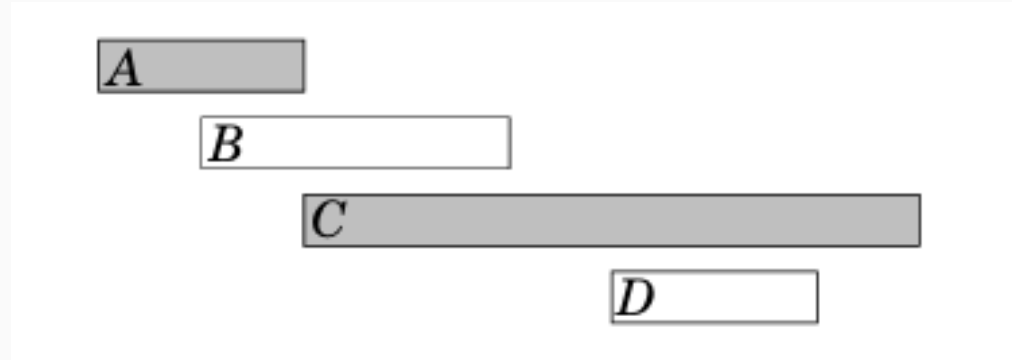
On propose de prendre à chaque fois, **l'événement le plus court disponible**.

Ceci dit, cette stratégie n'est pas toujours optimale, en voici un contre-exemple :



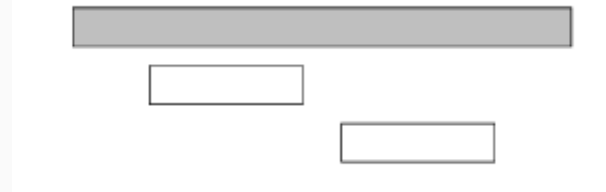
Si on prend le plus petit événement, on passe à côté des 2 autres événements qui peuvent s'enchaîner.

2.4 Problème A - Solution 2



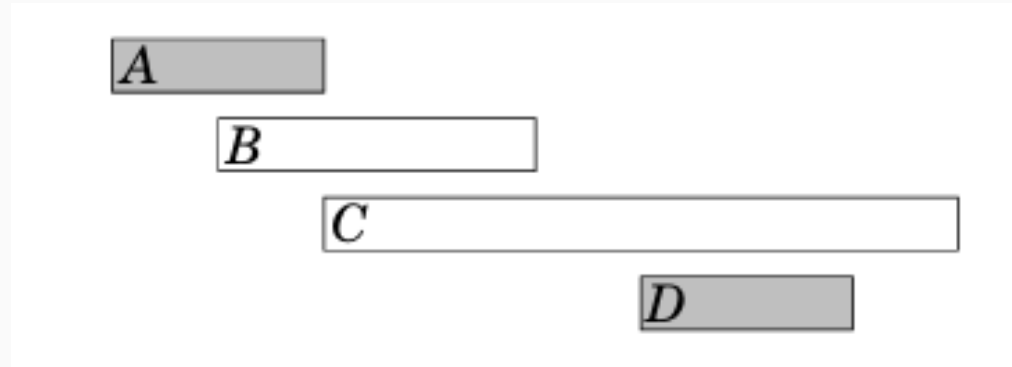
On propose de prendre à chaque fois, **le premier événement disponible**.

Comme la dernière solution, cette stratégie n'est pas optimale :



Si on prend le premier événement, les 2 autres qui peuvent être enchainés sont oubliés car nous sommes en train de réaliser l'événement grisé.

2.5 Problème A - Solution 3 (la bonne)



On propose de prendre à chaque fois, **l'événement qui finit le plus tôt possible**.

Et vraisemblablement ... c'est la bonne stratégie !

Pour justifier pourquoi c'est le cas, en prenant l'événement qui finit le plus tôt possible, nous avons plus de flexibilité sur les événements qui suivent.

2.6 Problème B

On s'intéresse maintenant au problème suivant :

Problème B

On a n tâches à réaliser avec une durée et une deadline pour chacun, le score d'une tâche est $d - x$ où d est la deadline et x quand la tâche finie. Quel est le score maximal ?

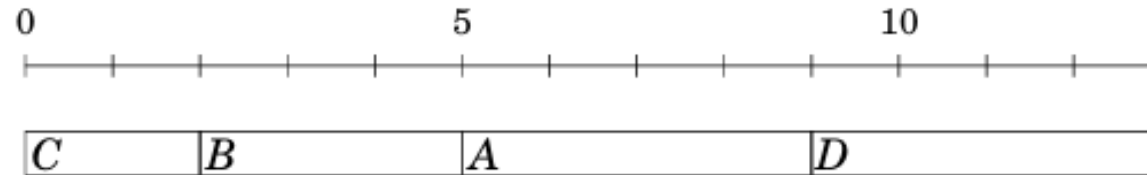
Nous allons prendre un exemple, avec la liste de tâches suivante:

Tâche	Durée	Deadline
A	4	2
B	3	5
C	2	7
D	4	5

2.7 Problème B - Solution

Il se trouve que la solution ne dépend pas du tout de la deadline ...

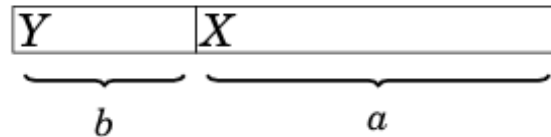
Voici la solution à l'exemple de la slide précédente :



On remarque alors qu'il est plus intéressant de traiter les tâches par leur durée dans l'ordre croissant.

2.7 Problème B - Solution

Si on a 2 tâches Y et X tel que la durée de X (a) est plus grande que celle de Y (b), si Y est faite après X , il est plus intéressant d'échanger l'ordre de ces 2 tâches.



En inversant les 2 tâches, Y donne a score en plus et X en donne b de moins, soit un gain de $a - b > 0$.

3. Crédits

Auteurs et images : Bilel MAHDJOUR et [CPH book](#)

Pour le **Bureau des technologies** de **Polytech Nice Sophia**

4. Sources et ressources

- [CPH book](#)
- [USACO guide - Greedy algorithms with sorting](#)